

Name – Jyotsna Kasibhotla

Roll no – 23102B0078

Community Health Survey System

DevOps Mini Project – Week 6 Report

Authentication, Authorization, User Management, Workflow Enforcement, and Dashboard Integration

Project Overview

Project Name: Community Health Survey System

Technology Stack: Spring Boot 3.2.5, Java 17, Maven, Thymeleaf, Spring Data JPA/Hibernate, MySQL

Project Path: community-health-survey/chss

Database: MySQL – health_survey_db

Application Port: 8081

1. Week 6 Objective

The primary objective of Week 6 was to introduce authentication and role-based authorization into the Community Health Survey System, and to integrate the existing user management framework with the survey module. This included enforcing a controlled survey status workflow, attributing survey-related actions to the authenticated user rather than to a hardcoded identity, and providing a dashboard that presents a real-time summary of survey data. Together, these enhancements transformed the application from a functionally complete but unsecured system into a secured, multi-role platform suitable for real-world use by different categories of users.

2. Authentication

Authentication was implemented using Spring Security to ensure that only registered users can access the application's protected functionality. The following authentication features were completed in Week 6:

- A dedicated login page was implemented for user sign-in.
- Authentication is performed using a username and password combination.
- Spring Security manages the authentication process, including session handling and access control.
- User passwords are stored securely in the database using BCrypt hashing; plain-text passwords are never persisted.
- When invalid credentials are submitted, the application displays a generic login error message, avoiding disclosure of whether the username or password was incorrect.
- Logout functionality is provided, allowing users to safely terminate their authenticated session.

3. User Roles

The application implements three distinct user roles, each representing a different category of system user:

- SURVEYOR – Responsible for creating and submitting community health surveys.
- ADMINISTRATOR / ADMIN – Responsible for overall system administration and management of survey status transitions.

- HEALTH_OFFICER – Responsible for reviewing and verifying submitted survey data.

Each role is granted a specific set of permissions strictly in line with the functionality implemented in the current application, as detailed in Section 6 (Role-Based Authorization).

4. User Entity and Database

User management is backed by a dedicated User entity and its corresponding repository, integrated with the existing survey data model.

4.1 User Entity

The User entity represents an application user and stores the username, hashed password, and assigned role. Each username is enforced as unique at the database level to prevent duplicate accounts.

4.2 User Repository

A UserRepository interface, built on Spring Data JPA, provides database access operations for the User entity, including lookup of users by username for authentication purposes.

4.3 Relationship Between Users and Surveys

The survey entity maintains a relationship to the User entity through a created_by association, linking each survey to the user who created it. This relationship enables accurate attribution of surveys to their creators.

4.4 Database Tables

Hibernate, operating through Spring Data JPA, automatically manages the following core tables based on the entity definitions:

- users – stores user accounts, hashed passwords, and roles.
- surveys – stores survey records, including a created_by foreign key referencing the users table.

All persistence operations are carried out against the MySQL database health_survey_db.

5. Predefined/Demo Users

To support testing and demonstration of the role-based system, the application initializes a set of predefined demo users at startup, one for each implemented role:

- surveyor1 – assigned the SURVEYOR role.
- admin1 – assigned the ADMINISTRATOR / ADMIN role.
- officer1 – assigned the HEALTH_OFFICER role.

These accounts are created through the application's data initialization logic, and their passwords are processed through the same BCrypt hashing mechanism used for all user accounts. In accordance with security best practice, actual passwords and credentials are not disclosed in this report.

6. Role-Based Authorization

Spring Security enforces role-based access control across the application's protected endpoints, ensuring that each role can only access functionality appropriate to its responsibilities.

Role	Available Functions
Surveyor	Create new surveys; view surveys they have created; submit surveys for review.

Role	Available Functions
Administrator	Access administrative functions; manage survey status transitions; view the dashboard and overall survey data.
Health Officer	Review submitted surveys; verify survey data as part of the status workflow.

Access to protected endpoints is restricted through Spring Security's security configuration, which maps roles to the corresponding permitted operations. Any request to a protected endpoint made without appropriate authentication or role authorization is denied.

7. Survey Creator Attribution

Prior to Week 6, survey creation did not associate surveys with a specific user. This has been corrected so that when a survey is created, the identity of the currently authenticated user is retrieved from the security context and stored against the survey, rather than relying on a hardcoded creator value.

Database Relationship

The relationship between users and surveys is implemented as follows:

- users (1) → surveys (many): a single user may create multiple surveys.
- The surveys table stores a `created_by` column, implemented as a foreign key referencing the `id` column of the users table.

This ensures that every survey record can be reliably traced back to the user who created it.

8. Survey Status Workflow

A controlled status workflow was implemented to govern the lifecycle of a survey record, ensuring that surveys progress through a defined and auditable sequence of states:

DRAFT → SUBMITTED → VERIFIED → CLOSED

- Every new survey is created with an initial status of DRAFT.
- Status transitions follow the defined sequence, and the application controls how a survey moves from one status to the next.
- Invalid or backward status transitions (for example, moving directly from DRAFT to CLOSED, or from a later status back to an earlier one) are rejected by the application.
- Where implemented, status changes are associated with the authenticated user who performed the action, supporting accountability for workflow transitions.
- Administrator permissions are applied to status-changing operations in accordance with the security rules configured in the application, restricting who may progress a survey through the workflow.

9. Dashboard

A dashboard was implemented to give users a real-time overview of survey activity within the system. The dashboard displays the following metrics, each retrieved directly from the database and reflecting the current state of survey data:

- Total number of surveys recorded in the system.
- Number of surveys currently in DRAFT status.
- Number of surveys currently in SUBMITTED status.
- Number of surveys currently in VERIFIED status.

- Number of surveys currently in CLOSED status.

These counts are computed dynamically from the surveys table, ensuring that the dashboard always presents an up-to-date summary of survey progress across the system.

10. Database Integration

Week 6 confirmed successful integration between the Spring Boot application and the MySQL database health_survey_db, using the following technologies:

- MySQL as the relational database management system.
- Spring Data JPA as the persistence abstraction layer.
- Hibernate as the underlying ORM (Object-Relational Mapping) implementation.

Hibernate is configured to automatically create and update the required database schema at application startup, based on the defined entity classes. This includes the users table and the surveys table, along with the foreign-key relationship between surveys.created_by and users.id described in Section 7. No database credentials or connection secrets are included in this report.

11. Configuration / Issue Resolution

During Week 6 development, the following configuration issues were encountered and successfully resolved:

11.1 MySQL Public Key Retrieval Error

On initial connection attempts, MySQL produced a "Public Key Retrieval is not allowed" error. This was resolved by correcting the JDBC connection configuration to permit public key retrieval, after which the application connected to MySQL successfully.

11.2 Port Conflict on 8080

The default Spring Boot port, 8080, was found to be already occupied by another Windows service on the development machine. To resolve this, the application was reconfigured to run on port 8081, allowing it to start without conflict.

Both issues were development/configuration matters and were resolved successfully prior to the completion of Week 6.

12. Validation and Testing

The following functional tests were carried out to verify the Week 6 implementation:

Test Case	Expected Result	Actual Result	Status
Valid login	User is authenticated and granted access to the application	User was authenticated and access was granted	Pass
Invalid login	A generic login error is displayed without revealing credential details	Generic login error was displayed as expected	Pass
Role-based access	Each role can access only its permitted functionality	Access was correctly restricted according to role	Pass
Survey creation	A created survey is attributed to the authenticated user and starts in DRAFT status	Survey was created with correct creator attribution and DRAFT status	Pass

Test Case	Expected Result	Actual Result	Status
Survey status workflow	Status progresses only in the defined order; invalid transitions are rejected	Valid transitions succeeded; invalid/backward transitions were rejected	Pass
Dashboard counts	Dashboard reflects accurate, current survey counts by status	Dashboard counts matched the underlying database records	Pass
MySQL persistence	Data created in the application is correctly persisted to MySQL	Data was persisted and retrievable from the health_survey_db database	Pass
Application startup	Application starts successfully on the configured port with schema created/updated	Application started successfully on port 8081 with schema applied	Pass

Note: Automated browser-based (Selenium) testing is outside the scope of Week 6 and is planned for a later stage of the project.

13. Week 6 Acceptance Criteria

Acceptance Criterion	Status
Users can log in using a username and password	Completed
Passwords are stored securely using hashing	Completed
Invalid login attempts show a generic error message	Completed
Users can log out of the application	Completed
Three user roles (Surveyor, Administrator, Health Officer) are implemented	Completed
Role-based access control restricts protected functionality	Completed
Surveys are attributed to the authenticated creator	Completed
Survey status workflow (DRAFT → SUBMITTED → VERIFIED → CLOSED) is enforced	Completed
Dashboard displays accurate survey counts by status	Completed
Application is integrated with MySQL via Spring Data JPA/Hibernate	Completed
Development/configuration issues are identified and resolved	Completed

14. Files / Components Implemented

The following classes and components were implemented or modified as part of the Week 6 work:

- User entity – represents an application user, including username, hashed password, and role.
- UserRepository – Spring Data JPA repository providing database access for user accounts.
- Security configuration – Spring Security configuration class defining authentication and role-based access rules.
- UserDetailsService implementation – loads user details for Spring Security during authentication.
- Authentication/Login controller and login view – handle the login page and authentication flow.
- Data initializer – seeds the predefined demo users (surveyor1, admin1, officer1) at application startup.
- Survey entity, service, and controller – updated to include creator attribution (created_by) and status workflow logic.
- Dashboard controller and view – compute and display survey counts by status.

15. Security Improvements

Week 6 introduced the following security improvements to the Community Health Survey System:

- BCrypt password hashing ensures that user passwords are never stored in plain text.
- Role-based authorization restricts application functionality according to the authenticated user's assigned role.
- Protected endpoints are secured so that they cannot be accessed without proper authentication and authorization.
- Users must authenticate before accessing any protected functionality within the application.
- A generic login failure message is displayed on invalid login attempts, avoiding disclosure of specific credential errors.
- Survey creation and status-changing actions are attributed to the authenticated user rather than relying on a hardcoded identity, improving accountability and traceability.

16. Final Outcome

Week 6 successfully delivered authentication, role-based authorization, and user management for the Community Health Survey System. The survey module was integrated with the user system through creator attribution, and a controlled status workflow (DRAFT → SUBMITTED → VERIFIED → CLOSED) was enforced to govern survey progress. A dashboard was implemented to provide a real-time summary of survey data, and full integration with the MySQL database was verified, including resolution of the configuration issues encountered during development. Together, these deliverables meet the objectives set for this stage of the project and establish a secure, role-aware foundation for subsequent phases.

Continuous Integration/Continuous Deployment (CI/CD) using Jenkins is planned as part of the next phase of the project and is not included in the scope of this Week 6 report.